Faculdade de Tecnologia de Votorantim

Ciência de Dados para dados e negócios

Lista 3- Utilização do colab : matrizes, determinantes e sistemas lineares

Aluna: Ana Elisa Rubinato Cavalheiro

1- Crie uma matriz quadrada de ordem 2.

```
[2] import numpy as np

    # Cria uma matriz quadrada de ordem 2 com valores inteiros aleatórios entre 1 e 10
    matriz = np.random.randint(1, 11, size=(2, 2))

    # Imprime a matriz
    print(matriz)
```

```
[[2 6]
 [6 2]]
```

2- Crie uma matriz quadrada de ordem 3

```
[3] # Cria uma matriz quadrada de ordem 3 com valores específicos
    matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

    # Imprime a matriz
    print(matriz)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

3- Crie uma matriz quadrada de ordem 4

```
[6] # Cria uma matriz de ordem 4 com valores inteiros aleatórios entre 1 e 20
    matriz = np.random.randint(1, 21, size=(4, 4))

    # Imprime a matriz
    print(matriz)
```

```
[[20  7 16  9]
 [15  1 20 16]
 [10 12  8 14]
 [10  3 18  2]]
```

4- Obtenha a matriz identidade de ordem 4.

```
[5]  # Obtém a matriz identidade de ordem 4
     matriz_identidade = np.identity(4)

     # Imprime a matriz identidade
     print(matriz_identidade)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

5- Aponte (se houver) diferenças entre o caminho para a programação de uma matriz de ordem 3 e outra de ordem 4.

Para entender melhor imagine uma matriz como uma tabela de números organizada em linhas e colunas, onde a "ordem" indica quantas linhas e colunas ela possui. Uma matriz de ordem 3 é como uma tabela quadrada 3x3, enquanto uma de ordem 4 é uma tabela quadrada 4x4, sendo maior e contendo mais números. Essa diferença de tamanho acarreta em mais trabalho para o computador ao realizar cálculos, como somar, multiplicar ou encontrar valores específicos dentro da matriz. Assim como um tabuleiro de xadrez maior exige mais tempo para somar todos os seus números, uma matriz de ordem 4 demanda mais processamento e memória do computador do que uma de ordem 3. Além disso, algumas operações matemáticas, como o cálculo do determinante, tornam-se consideravelmente mais complexas à medida que a ordem da matriz aumenta. Em resumo, a principal distinção entre programar matrizes de ordem 3 e 4 reside no tamanho, o que impacta diretamente na complexidade dos cálculos, na quantidade de memória necessária e no tempo de processamento.

6- Obtenha uma matriz e seu determinante.

```
# Cria uma matriz quadrada de ordem 2 com valores inteiros aleatórios entre 1 e 10
matriz = np.random.randint(1, 11, size=(2, 2))

# Imprime a matriz
print("Matriz:")
print(matriz)
```

```
Matriz:
[[ 4  2]
 [10  4]]
```

```
[8]  # Calcula o determinante da matriz
     determinante = np.linalg.det(matriz)

     # Imprime o determinante
     print("\nDeterminante:")
     print(determinante)
```

```

Determinante:
-4.0
```

7- Crie uma matriz cujo determinante é nulo.

```
[9]  # Cria uma matriz de ordem 3 com determinante nulo
     matriz = np.array([[1, 2, 3], [2, 4, 6], [4, 5, 6]])

     # Imprime a matriz
     print("Matriz:")
     print(matriz)

     # Calcula o determinante da matriz
     determinante = np.linalg.det(matriz)

     # Imprime o determinante
     print("\nDeterminante:")
     print(determinante)
```

```
Matriz:
[[1 2 3]
 [2 4 6]
 [4 5 6]]

Determinante:
0.0
```

8- Dados a matriz |A e um vetor b abaixo, obter um código para a resolução do sistema de equações lineares, usando o método do escalonamento.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{pmatrix}, \quad b = \begin{pmatrix} 9 \\ 13 \\ 17 \end{pmatrix}$$

```
[10] A = np.array([[1, 2, 3], [2, 3, 4], [3, 4, 5]], dtype=float)
     b = np.array([9, 13, 17], dtype=float)

     # Matriz aumentada [A|b]
     Ab = np.concatenate((A, b.reshape(-1, 1)), axis=1)

     # Escalonamento
     Ab[1] = Ab[1] - 2 * Ab[0]
     Ab[2] = Ab[2] - 3 * Ab[0]
     Ab[2] = Ab[2] - 2 * Ab[1]

     # Solução
     print("Matriz escalonada:")
     print(Ab)

     # Como o sistema tem infinitas soluções, podemos expressá-las em função de z
     print("\nSolução:")
     print("x = -1 + z")
     print("y = 5 - 2z")
     print("z = z")
```

```
Matriz escalonada:
[[ 1.  2.  3.  9.]
 [ 0. -1. -2. -5.]
 [ 0.  0.  0.  0.]]

Solução:
x = -1 + z
y = 5 - 2z
z = z
```

Em seguida, veja se consegue identificar pelo menos 3 operações elementares, que levaram á resolução do sistema. Qual é a solução deste sistema?

```python
def escalonamento(A, b):

    # 1ª Operação: Cria a matriz aumentada [A|b] combinando A e b.
    Ab = np.concatenate((A, b.reshape(-1, 1)), axis=1).astype(float)

    # Obtém o número de equações (linhas) do sistema.
    n = len(b)

    # Loop principal para escalonar a matriz.
    for i in range(n):
        # Loop para zerar os elementos abaixo do pivô na coluna i.
        for j in range(i + 1, n):
            # 2ª Operação: Calcula o fator para zerar o elemento Ab[j, i].
            fator = Ab[j, i] / Ab[i, i]

            # 3ª Operação: Subtrai a linha i multiplicada pelo fator da linha j.
            Ab[j] = Ab[j] - fator * Ab[i]

    # Cria o vetor solução x com zeros.
    x = np.zeros(n)

    # Loop para substituição retroativa (resolver o sistema triangular superior).
    for i in range(n - 1, -1, -1):
        # 4ª Operação: Calcula o valor de x[i] usando os valores já encontrados em x[i+1:].
        x[i] = (Ab[i, -1] - np.dot(Ab[i, i+1:-1], x[i+1:])) / Ab[i, i]

    # Retorna o vetor solução x.
    return x

# Matriz A dos coeficientes do sistema.
A = np.array([[1, 2, 3], [2, 3, 4], [3, 4, 5]])

# Vetor b dos termos independentes do sistema.
b = np.array([9, 13, 17])

# Resolve o sistema usando a função escalonamento.
solucao = escalonamento(A, b)

# Verifica se a solução contém NaN (Not a Number), indicando infinitas soluções.
if np.isnan(solucao).any():
    print("Sistema com infinitas soluções.")
else:
    # Imprime a solução encontrada.
    print("Solução:", solucao)

# Geração de 3 números aleatórios para ajudar na resolução (z = variável livre)
z_valores = [0, 1, 2]  # Três valores aleatórios para z

print("\nSoluções para z aleatórios:")
for z in z_valores:
    x = -1 + z
    y = 5 - 2 * z
    print(f"z = {z}, x = {x}, y = {y}")
```

```
Sistema com infinitas soluções.

Soluções para z aleatórios:
z = 0, x = -1, y = 5
z = 1, x = 0, y = 3
z = 2, x = 1, y = 1
```

9- Obter um código para a criação e resolução de um sistema Compatível e Indeterminado.

```python
[14] def criar_sistema_indeterminado(n_equacoes, n_incognitas):

        # Cria uma matriz A com posto menor que o número de incógnitas
        A = np.random.rand(n_equacoes, n_incognitas)
        posto = np.linalg.matrix_rank(A)
        while posto == n_incognitas:
            A = np.random.rand(n_equacoes, n_incognitas)
            posto = np.linalg.matrix_rank(A)

        # Cria um vetor b compatível com a matriz A
        b = np.random.rand(n_equacoes)

        return A, b


    def resolver_sistema_indeterminado(A, b):
        # Resolve um sistema linear compatível e indeterminado.
        try:
            # Resolve o sistema usando a função lstsq (mínimos quadrados)
            solucao, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
            return solucao
        except np.linalg.LinAlgError:
            return None


    # Exemplo de uso
    n_equacoes = 2
    n_incognitas = 3

    A, b = criar_sistema_indeterminado(n_equacoes, n_incognitas)

    print("Matriz A:")
    print(A)
    print("\nVetor b:")
    print(b)

    solucao = resolver_sistema_indeterminado(A, b)

    if solucao is not None:
        print("\nSolução:")
        print(solucao)
    else:
        print("\nO sistema não tem solução.")
```

```
Matriz A:
[[0.3434223  0.12356111 0.40360823]
 [0.49206906 0.8841806  0.57045536]]

Vetor b:
[0.00638383 0.02242621]

Solução:
[0.00505267 0.01884265 0.00574917]
```

10- Obter a resolução do sistema linear abaixo:

$$\begin{cases} 3x - y + z = 11 \\ 2x + y - z = -1 \\ x + 2y - 3z = -12 \end{cases}$$

```python
A = np.array([[3, -1, 1], [2, 1, -1], [1, 2, -3]])
b = np.array([11, -1, -12])

solucao = np.linalg.solve(A, b)

print("Solução:")
print(solucao)
```

```
Solução:
[ 2. -1.  4.]
```

11- Um vendedor de camisetas e bonés em um evento esportivo vendeu 3 camisetas e 2 bonés, arrecadando um total de R$ 220,00. No dia seguinte, ele vendeu 2 camisetas e 3 bonés, arrecadando R$ 190,00. Qual seria o preço de uma camiseta e o preço de um boné? Monte o sistema aqui antes de ir para o Colab.

Variáveis:

- **x:** preço de uma camiseta
- **y:** preço de um boné

Equações:

- **Dia 1:** 3 camisetas + 2 bonés = R$ 220,00 -> 3x + 2y = 220
- **Dia 2:** 2 camisetas + 3 bonés = R$ 190,00 -> 2x + 3y = 190

Sistema de Equações Lineares:

```
3x + 2y = 220
2x + 3y = 190
```

```python
# Matriz A dos coeficientes do sistema
A = np.array([[3, 2], [2, 3]])

# Vetor b dos termos independentes do sistema
b = np.array([220, 190])

# Resolve o sistema usando a função np.linalg.solve()
solucao = np.linalg.solve(A, b)

# Imprime a solução
print("Preço da camiseta: R$ {:.2f}".format(solucao[0]))
print("Preço do boné: R$ {:.2f}".format(solucao[1]))
```

```
Preço da camiseta: R$ 56.00
Preço do boné: R$ 26.00
```

12- Uma loja vende camisetas, bermudas e sapatos. No primeiro dia, foram vendidas 2 camisetas, 3 bermudas e 4 pares de sapatos, totalizando R$ 350,00. No segundo dia, foram vendidas 3 camisetas, 2 bermudas e 1 par de sapatos, totalizando R$ 200,00. No terceiro dia, foram vendidas 1 camiseta, 4 bermudas e 2 pares de sapatos, totalizando R$ 320,00. Qual seria o preço de uma camiseta, uma bermuda e um par de sapatos? Monte o sistema aqui antes de ir para o Colab.

Variáveis:

- **x:** preço de uma camiseta
- **y:** preço de uma bermuda
- **z:** preço de um par de sapatos

Equações:

- **Dia 1:** 2 camisetas + 3 bermudas + 4 pares de sapatos = R$ 350,00 -> 2x + 3y + 4z = 350

- **Dia 2:** 3 camisetas + 2 bermudas + 1 par de sapatos = R$ 200,00 -> 3x + 2y + z = 200
- **Dia 3:** 1 camiseta + 4 bermudas + 2 pares de sapatos = R$ 320,00 -> x + 4y + 2z = 320

Sistema de Equações Lineares:

```
2x + 3y + 4z = 350
3x + 2y + z = 200
x + 4y + 2z = 320
```

```
# Matriz A dos coeficientes do sistema
A = np.array([[2, 3, 4], [3, 2, 1], [1, 4, 2]])

# Vetor b dos termos independentes do sistema
b = np.array([350, 200, 320])

# Resolve o sistema usando a função np.linalg.solve()
solucao = np.linalg.solve(A, b)

# Imprime a solução
print("Preço da camiseta: R$ {:.2f}".format(solucao[0]))
print("Preço da bermuda: R$ {:.2f}".format(solucao[1]))
print("Preço do par de sapatos: R$ {:.2f}".format(solucao[2]))
```

```
Preço da camiseta: R$ 16.00
Preço da bermuda: R$ 58.00
Preço do par de sapatos: R$ 36.00
```

13- Um restaurante oferece três opções de prato: carne, salada e pizza. No primeiro dia, foram vendidos 40 pratos de carne, 30 pratos de salada e 10 pizzas, totalizando R$ 7000,00 em vendas. No segundo dia, foram vendidos 20 pratos de carne, 40 pratos de salada e 30 pizzas, totalizando R$ 6000,00 em vendas. No terceiro dia, foram vendidos 10 pratos de carne, 20 pratos de salada e 40 pizzas, totalizando R$ 5000,00 em vendas. Qual seria o preço de cada prato? Monte o sistema aqui antes de ir para o Colab.

Variáveis:

- **x:** preço do prato de carne
- **y:** preço do prato de salada
- **z:** preço da pizza

Equações:

- **Dia 1:** 40 pratos de carne + 30 pratos de salada + 10 pizzas = R$ 7000,00 -> 40x + 30y + 10z = 7000
- **Dia 2:** 20 pratos de carne + 40 pratos de salada + 30 pizzas = R$ 6000,00 -> 20x + 40y + 30z = 6000
- **Dia 3:** 10 pratos de carne + 20 pratos de salada + 40 pizzas = R$ 5000,00 -> 10x + 20y + 40z = 5000

Sistema de Equações Lineares:

```
40x + 30y + 10z = 7000
20x + 40y + 30z = 6000
10x + 20y + 40z = 5000
```

```python
# Matriz A dos coeficientes do sistema
A = np.array([[40, 30, 10], [20, 40, 30], [10, 20, 40]])

# Vetor b dos termos independentes do sistema
b = np.array([7000, 6000, 5000])

# Resolve o sistema usando a função np.linalg.solve()
solucao = np.linalg.solve(A, b)

# Imprime a solução
print("Preço do prato de carne: R$ {:.2f}".format(solucao[0]))
print("Preço do prato de salada: R$ {:.2f}".format(solucao[1]))
print("Preço da pizza: R$ {:.2f}".format(solucao[2]))
```

```
Preço do prato de carne: R$ 140.00
Preço do prato de salada: R$ 20.00
Preço da pizza: R$ 80.00
```

14- Resolva o sistema de equações abaixo

a) $\begin{cases} 3x - y + z = 11 \\ 2x + y - z = -1 \\ x + 2y - 3z = -12 \end{cases}$ .Aponte as operações(pelo menos 3) elementares utilizadas pelo Numpy.

```python
A = np.array([[3, -1, 1], [2, 1, -1], [1, 2, -3]])
b = np.array([11, -1, -12])

solucao = np.linalg.solve(A, b)

print("Solução:")
print(solucao)
```

```
Solução:
[ 2. -1.  4.]
```

```python
import numpy as np

# Matriz de exemplo
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 1. Troca de linhas (troca a primeira e a segunda linha)
A[[0, 1]] = A[[1, 0]]
print("Troca de linhas:\n", A)

# 2. Multiplicação de uma linha por um escalar (multiplica a segunda linha por 2)
A[1] = A[1] * 2
print("\nMultiplicação de linha por escalar:\n", A)

# 3. Adição de um múltiplo de uma linha a outra linha (adiciona a primeira linha à terceira linha)
A[2] = A[2] + A[0]
print("\nAdição de múltiplo de linha a outra:\n", A)
```

```
Troca de linhas:
 [[4 5 6]
 [1 2 3]
 [7 8 9]]

Multiplicação de linha por escalar:
 [[4 5 6]
 [2 4 6]
 [7 8 9]]

Adição de múltiplo de linha a outra:
 [[ 4  5  6]
 [ 2  4  6]
 [11 13 15]]
```

b) $\begin{cases} x + y - z = 1 \\ 3x - 2y + 7z = 3 \\ 4x - y + 6z = 4 \end{cases}$ Aponte as operações (pelo menos 3) elementares utilizadas pelo Numpy

```python
import numpy as np
# Matriz dos coeficientes A
A = np.array([[1, 1, -1],
              [3, -2, 7],
              [4, -1, 6]])

# Vetor dos termos independentes b
b = np.array([1, 3, 4])

# Matriz aumentada [A|b]
Ab = np.concatenate((A, b.reshape(-1, 1)), axis=1)

# Calcular o posto da matriz A
posto_A = np.linalg.matrix_rank(A)
print("Posto da matriz A:", posto_A)

# Calcular o posto da matriz aumentada Ab
posto_Ab = np.linalg.matrix_rank(Ab)
print("Posto da matriz aumentada Ab:", posto_Ab)

# Número de incógnitas
n_incognitas = A.shape[1]
print("Número de incógnitas:", n_incognitas)

# Verificar se o sistema é possível e indeterminado
if posto_A == posto_Ab and posto_A < n_incognitas:
    print("\nO sistema é possível e indeterminado.")
    print("Solução geral (em função de z):")
    print("x = 1 - z")
    print("y = 2z")
    print("z = z")

    # Gerar algumas soluções particulares para demonstração
    print("\nAlgumas soluções particulares:")
    for z_val in [0, 1, -1, 2]:
        x_val = 1 - z_val
        y_val = 2 * z_val
        print(f"Para z = {z_val}: x = {x_val}, y = {y_val}")
elif posto_A == posto_Ab and posto_A == n_incognitas:
    try:
        solucao = np.linalg.solve(A, b)
        print("\nO sistema tem solução única:")
        print("x =", solucao[0])
        print("y =", solucao[1])
        print("z =", solucao[2])
    except np.linalg.LinAlgError:
        print("\nO sistema não tem solução (matriz singular).")
else:
    print("\nO sistema é impossível (não tem solução).")
```

```
Posto da matriz A: 2
Posto da matriz aumentada Ab: 2
Número de incógnitas: 3

O sistema é possível e indeterminado.
Solução geral (em função de z):
x = 1 - z
y = 2z
z = z

Algumas soluções particulares:
Para z = 0: x = 1, y = 0
Para z = 1: x = 0, y = 2
Para z = -1: x = 2, y = -2
Para z = 2: x = -1, y = 4
```

```python
import numpy as np

# Matriz dos coeficientes A
A = np.array([[1, 1, -1],
              [3, -2, 7],
              [4, -1, 6]])

# Vetor dos termos independentes b
b = np.array([1, 3, 4])

# Matriz aumentada [A|b]
Ab = np.concatenate((A, b.reshape(-1, 1)), axis=1)

print("Matriz A original:")
print(A)
print("\nMatriz aumentada Ab original:")
print(Ab)

# Operação Elementar 1: Trocar a linha 0 com a linha 1 (não necessária para este sistema, mas demonstração)
A_trocada = A[[1, 0, 2]]
Ab_trocada = Ab[[1, 0, 2]]
print("\nMatriz A após trocar linha 0 e 1:")
print(A_trocada)
print("\nMatriz aumentada Ab após trocar linha 0 e 1:")
print(Ab_trocada)

# Operação Elementar 2: Multiplicar a linha 0 por um escalar (-3) e somar à linha 1
fator = -3
A_op2 = A.copy() # Criar uma cópia para não modificar o original
Ab_op2 = Ab.copy()
A_op2[1] = A_op2[1] + fator * A_op2[0]
Ab_op2[1] = Ab_op2[1] + fator * Ab_op2[0]
print("\nMatriz A após multiplicar linha 0 por -3 e somar à linha 1:")
print(A_op2)
print("\nMatriz aumentada Ab após multiplicar linha 0 por -3 e somar à linha 1:")
print(Ab_op2)

# Operação Elementar 3: Multiplicar a linha 0 por um escalar (-4) e somar à linha 2
fator = -4
A_op3 = A.copy()
Ab_op3 = Ab.copy()
A_op3[2] = A_op3[2] + fator * A_op3[0]
Ab_op3[2] = Ab_op3[2] + fator * Ab_op3[0]
print("\nMatriz A após multiplicar linha 0 por -4 e somar à linha 2:")
print(A_op3)
print("\nMatriz aumentada Ab após multiplicar linha 0 por -4 e somar à linha 2:")
print(Ab_op3)
```

```
Matriz A original:
[[ 1  1 -1]
 [ 3 -2  7]
 [ 4 -1  6]]

Matriz aumentada Ab original:
[[ 1  1 -1  1]
 [ 3 -2  7  3]
 [ 4 -1  6  4]]

Matriz A após trocar linha 0 e 1:
[[ 3 -2  7]
 [ 1  1 -1]
 [ 4 -1  6]]
```


```
Matriz A original:
[[ 1  1 -1]
 [ 3 -2  7]
 [ 4 -1  6]]

Matriz aumentada Ab original:
[[ 1  1 -1  1]
 [ 3 -2  7  3]
 [ 4 -1  6  4]]

Matriz A após trocar linha 0 e 1:
[[ 3 -2  7]
 [ 1  1 -1]
 [ 4 -1  6]]

Matriz aumentada Ab após trocar linha 0 e 1:
[[ 3 -2  7  3]
 [ 1  1 -1  1]
 [ 4 -1  6  4]]
```

```
Matriz A após multiplicar linha 0 por -3 e somar à linha 1:
[[ 1  1 -1]
 [ 0 -5 10]
 [ 4 -1  6]]

Matriz aumentada Ab após multiplicar linha 0 por -3 e somar à linha 1:
[[ 1  1 -1  1]
 [ 0 -5 10  0]
 [ 4 -1  6  4]]

Matriz A após multiplicar linha 0 por -4 e somar à linha 2:
[[ 1  1 -1]
 [ 3 -2  7]
 [ 0 -5 10]]

Matriz aumentada Ab após multiplicar linha 0 por -4 e somar à linha 2:
[[ 1  1 -1  1]
 [ 3 -2  7  3]
 [ 0 -5 10  0]]
```